

BONUS TRACK · 심화 세션

견적서의 나머지 항목

원칙 4 개로 못 줄인 비용은 어디 있나

캐싱 · 에이전트 루프 · SDD · 라우팅 · 배치 · 압축 — 그리고 240 배 실측

본편이 “무엇을 시킬까”였다면, 보너스는 “어디서 돌릴까”다. 같은 요청도 실행 위치를 바꾸면 요금이 바뀐다.

본편 22 장을 먼저 보고 오세요. 이 텍은 그 위에 쌓는 레버 모음입니다.

보너스 ⑤ 프롬프트 캐싱 — 입력 자리를 더 싸게

제공사	캐시 읽기 단가	조건
Anthropic	0.10× (90% 할인)	명시적 marker, write 1.25×
OpenAI	0.50× (50% 할인)	자동, 1,024 토큰 이상
Google	0.25× (75% 할인)	implicit / explicit
DeepSeek	0.10× (90% 할인)	자동

실측 시뮬레이션

고정 컨텍스트 20K + 가변 1K,
출력 1.5K, 100 회 호출

캐시 없음 : \$8.55

캐시 적용 : \$3.22

→ 62% 절감 (품질 변화 0)

전제 조건 : 고정 컨텍스트를 프롬프트 “앞쪽”에 두고 순서를 바꾸지 말 것 .

시스템 프롬프트 → 톨 정의 → 레포 맵 → 스펙 → (여기까지 캐시) → 가변 질문 . 앞부분이 1 바이트라도 바뀌면 캐시 전체가 무효화된다 .

실사례 : 캐시 히트율 7% → 84% 개선으로 총 LLM 지출 59~70% 감소 (ProjectDiscovery).

캐싱 심화 — 히트율 계단과 안티패턴

캐시 히트율	100 콜 비용	캐시 미사용 대비
캐시 미사용 (기준)	\$8.55	
0% — 컷는데 매번 미스	\$10.05	+18%
40%	\$7.32	-14%
80%	\$4.59	-46%
95%	\$3.56	-58%
100% (이론 상한)	\$3.22	-62%

→ 리포의 exp07 진단기가 이 동적값을 자동으로 잡아준다.

손익분기 : 0.28 회

캐시 쓰기는 1.25 배 비싸지만 , 같은 프리픽스를 한 번만 더 재사용해도 즉시 이득이다 .

“재사용이 적어서 안 쓴다”는 거의 항상 오판 .

최악의 안티패턴 = 3.1 배

맨 앞에 타임스탬프 · 요청 ID · “현재 시각”을 넣으면 매 호출 캐시가 전부 깨진다 . 쓰기 프리미엄만 물어 캐시를 아예 안 쓴 것보다 18% 비싸진다 .

캐시 최적 \$3.22 → 안티패턴 \$10.05 = 3.1 배

에이전트 루프 — 비용은 선형이 아니라 2 차 함수다

매 턴 전체 대화가 다시 입력으로 들어간다 . 턴이 2 배면 비용은 약 3 배가 된다 (10→20 턴 3.1 배).

세션 길이	누적 비용	증가
5 턴	\$0.26	
10 턴	\$0.74	2.8×
20 턴	\$2.32	3.1×
30 턴	\$4.73	2.0×
50 턴	\$12.09	2.6×

20 턴 세션 , 전략별 비용

무대책	\$2.32
캐싱만	\$1.88 (-19%)
캐싱 + 10 턴마다 컴팩션	\$1.31 (-44%)
캐싱 + 5 턴마다 컴팩션	\$0.97 (-58%)

컴팩션 = 오래된 톨 결과 · 중간 로그를 요약으로
교체하는 것 . 100 턴 평가에서 84% 절감 보고 .

실무 규칙 : 작업이 끝나면 대화를 끊어라 . “ 하나의 대화로 종일 ”이 가장 비싼 습관이다 .

Anthropic: 에이전트는 챗 대비 4 배 , 멀티에이전트는 15 배 토큰을 쓴다 . 에이전트 지출의 99% 이상이 입력 토큰이다 .

스펙 주도 개발 (SDD) 은 조건부로만 참이다

유리한 쪽 — 자체 시뮬레이션

vibe 코딩 40 턴	\$1.75
규약 파일 적용	\$1.09 (-38%)
SDD (스펙 4K + 12 턴)	\$0.51 (-71%)

스펙이 턴 수를 줄이면, 2 차 함수 구간 자체가
잘려 나가기 때문에 절감폭이 크게 나온다.

불리한 쪽 — 외부 반증

Spec-Kit 은 OpenSpec 대비 토큰 +97~109%
(57,740 → 120,947 / 91,729 → 181,040).
무거운 SDD 프레임워크는 오히려 비용을 늘린다.

ETH Zurich (138 이슈, 4 개 모델): LLM 이 생성한
컨텍스트 파일은 성공률 소폭 하락 + 비용 20%↑.

결론 : “스펙을 쓰라”가 아니라 “가볍게, 결정 중심으로 쓰라”

엔드포인트 · 스키마 · 에러 · 완료조건처럼 모델이 추측하면 틀리는 것만 적는다. 코드를 읽으면 아는 것은 스펙에도 쓰지 않는다.

나머지 레버 4종 — 언제 쓰고, 언제 쓰지 말 것인가

배치 API

입력·출력 정액 50% 할인. 캐싱과 중복 적용 가능.

○ 야간 평가·벌크 분류·리포트 생성

× SLA 24 시간. 사용자 대면 채팅·툴 루프 불가.

시맨틱 캐싱

유사 질문을 임베딩으로 매칭해 호출 자체를 제거.

○ FAQ·고객지원·반복 질의 (히트율 20~45%)

× 벤더의 95% 주장은 과장. 임계값 0.8 권장.

모델 라우팅

쉬운 요청은 소형 모델로. RouteLLM 85%↓ @ 품질 95%.

○ 분류·추출·요약 등 난이도 편차 큰 트래픽

× 벤치마크 임계값 그대로 쓰면 과대추정. 재보정 필수.

프롬프트 압축

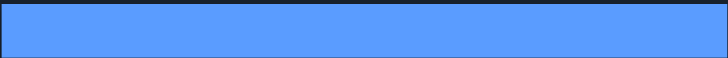
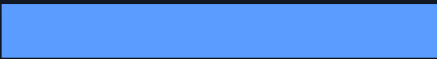
LLMLingua 최대 20x 압축, 손실 2% 미만.

○ RAG 컨텍스트·장황한 시스템 프롬프트·few-shot

× 코드·수식·짧은 쿼리에는 쓰지 말 것.

레버는 곱해진다 — 스택 적층 시뮬레이션

각 레버는 “앞 단계가 걸러낸 나머지”에 적용된다. 그래서 30% + 50% 는 80% 가 아니라 65% 다.

시작 — 무대책		100%	
① 시맨틱 캐싱 (호출 30% 제거)		70%	-30%p
② 모델 라우팅 (쉬운 건 소형으로)		42%	-28%p
③ 프롬프트 캐싱 (프리픽스 재사용)		21%	-21%p
④ verbosity / effort 조정		16%	-5%p
⑤ 배치 API (비대면 트래픽만)		14%	-2%p

누적 86% 절감 — 업계 보고치 60~80% 의 상한선에 해당한다. 전부 적용했을 때의 이론값으로 읽을 것.

현실적 목표: 상위 2~3 개만 제대로 해도 50~65%. 8 개를 어설프게 하는 것보다 낫다.

같은 정답률 , 240 배 요금 — 30 회 실측

같은 계산 문제 30 회 . 사고 예산만 변경 . 정답 \$38.016 · 오른쪽 열은 6 회 중 정답 횟수 .

사고 예산	n	사고 토큰	응답	비용 / 호출	배수	정답률 (±0.05)
미지정 (기본)	6	0	6	\$0.000155	1.0×	정답 6/6
꿈 (0)	6	0	6	\$0.000158	1.0×	정답 5/6 (1 회 380.16)
512	6	142	6	\$0.001570	10.1×	정답 2/6 ← 품질 붕괴
2048	6	1,114	6	\$0.011302	72.9×	정답 6/6
자동 (-1)	6	3,704	6	\$0.037203	240×	정답 6/6

사고를 끄든 켜든 정답률은 6/6 으로 같습니다 . 요금만 240 배 차이납니다 .

출력 토큰의 99% 가 보이지 않는다

사고가 켜진 구간에서 과금 출력의 99% 가 thoughtsTokenCount 다 (실측 99.6%).

예산 512 는 정답률이 2/6 으로 떨어졌다

어중간한 예산은 추론이 끊겨 품질까지 나빠진다 . 충분히 주거나 아예 끌 것 .

용어부터 — 라우팅 · 리워크 · 워크플로우 컨트롤

셋은 자주 섞여 쓰이지만, 줄이는 비용의 종류가 다르다. 섞으면 처방도 섞인다.

① 라우팅 (Routing)

질문 : “ 이 요청을 어느 모델에 보낼
까 ? ”

단위 : 요청 1 건 .

줄이는 것 : 과잉 스펙 모델에 쓰는 단가 .

효과 : -49~84 % (exp06).

② 리워크 (Rework)

질문 : “ 왜 같은 걸 다시 시켰나 ? ”

단위 : 턴의 반복 횟수 .

줄이는 것 : 실패한 시도의 재생성 .

효과 : 48 턴 리워크 = 16.6 배

(exp09-C).

③ 워크플로우 컨트롤

질문 : “ 이 일을 어떤 순서 · 형태로
돌릴까 ? ”

단위 : 작업 파이프라인 전체 .

줄이는 것 : 단계 배치의 낭비 .

②와 ①을 포함하는 상위 개념 .

오늘 다루는 것은 ③ 워크플로우 컨트롤이다 — 라우팅은 그 안의 한 수단이다 .

“작은 모델로 컨트롤한다”는 것은 값싼 모델이 지휘자를 맡고 , 비싼 모델은 꼭 필요한 구간에만 호출되는 구조를 뜻한다 . 리워크 감소는 그 결과로 따라온다 .

배치 처리 — “24 시간”은 상한이지 예정 시각이 아니다

제공자	할인	공식 표현	실무 관찰
OpenAI Batch	50%	24h 내 완료	수 분 ~2h 혼함
Gemini Batch	50%	목표 24h, 대개 더 빠름	작은 잡은 즉시급
Anthropic Batch	50%	대부분 1h 미만	캐싱과 중첩 가능
xAI Batch	50%	대개 24h 내	베스트 에포트

그러나 — 실패 모드가 있다

2025 년 초 OpenAI 배치 지연 사례 :
 평소 1~2h → 24h 근접 , 만료
 (expired) 비율이 0%대에서 25%↑로
 급증한 기간이 보고됨 .

즉 “보통 10 분”은 평균이지 보장이
 아니다 . 마감에 있는 작업에 배치를
 쓰려면 폴백 경로가 필요하다 .

규칙 : 배치는 “늦어도 되는 일”에만 . 늦으면 곤란한 일은 동기 호출로 풀백 .

적합 : 자료조사 · 정규화 · 라벨링 · 임베딩 · 평가 — 사람이 기다리지 않는 작업 . 부적합 : 대화 턴 중간 , 에이전트 루프 내부 (앞 결과가 다음 요청의 입력인 경우) .